トランジションセット検索モーダル UI仕様書

概要

目的

4軸ベクトル検索により、実験プロトコルのトランジションレベルでの精密な類似度評価を行う統合検索モーダルコンポーネント。自然言語クエリをAI分解し、入力条件・手順・出力・構造の4軸で検索する。

使用場面

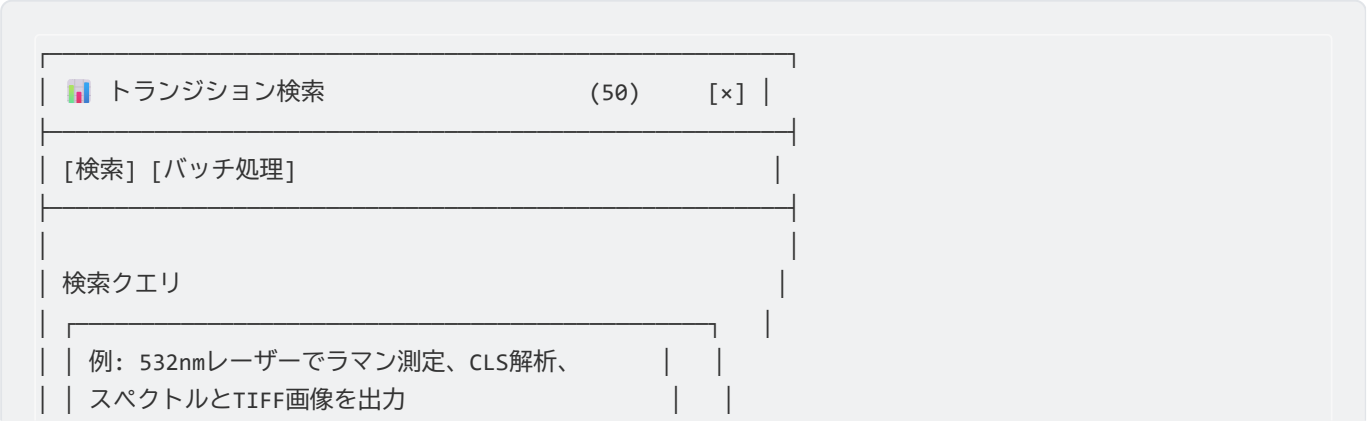
- プロジェクト管理画面のツールバーから「TransitionSet検索」ボタンをクリック
- 類似する実験プロトコルを検索する際に使用
- 過去の実験データからTransitionSetを検索・再利用

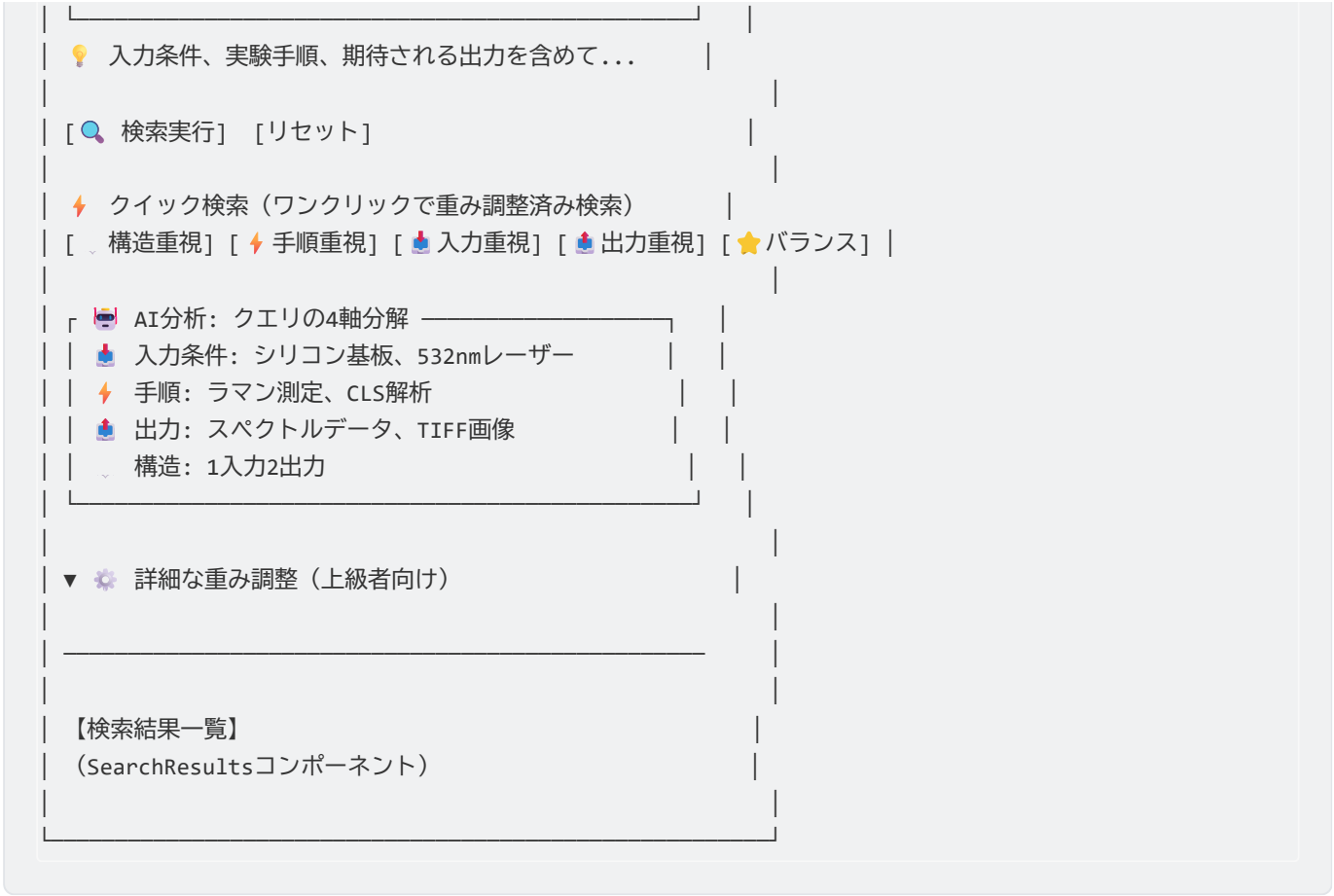
アクセス権限

- **Experiment Admin（実験管理者）**: TransitionSet検索の実行
- **その他のロール**: アクセス不可

画面構成

レイアウト構造





UI要素

1. モーダルヘッダー

- **アイコン**: 📊 `AccountBookOutlined`
- **タイトル**: 「トランジション検索」
- **バッジ**: 検索結果件数（青色、検索実行後のみ表示）

2. タブ

- **検索タブ**: メイン検索画面
- **バッチ処理タブ**: `CloudUploadOutlined` + 「バッチ処理」

3. 検索クエリ入力

- **ラベル**: 「検索クエリ」（12px）
- **入力欄**: TextArea、3行

- **プレースホルダー**: 「例: 532nmレーザーでラマン測定、CLS解析、スペクトルとTIFF画像を出力」
- **ヒント**: 「💡 入力条件、実験手順、期待される出力を含めて記述してください」（11px、グレー）
- **Enter動作**: Shift+Enter以外で検索実行

4. 検索ボタン

- **検索実行ボタン**: `SearchOutlined`、プライマリ、large、最小幅140px
  - 通常: 「検索実行」
  - 処理中: 「検索中...」（loading表示）
- **リセットボタン**: 「リセット」、default

5. クイック検索ボタン

ボタン	アイコン	ラベル	プリセットキー
構造重視	`ApartmentOutlined`	構造重視	structureFocused
手順重視	`ThunderboltOutlined`	手順重視	procedureFocused
入力重視	`InboxOutlined`	入力重視	inputFocused
出力重視	`ExportOutlined`	出力重視	outputFocused
バランス	`StarOutlined`	バランス	balanced

6. AI分解結果表示カード

- **背景色**: `#f9f0ff`（淡い紫）
- **ボーダー**: `1px solid #d3adf7`
- **タイトル**: 「🤖 AI分析: クエリの4軸分解」（12px、太字）
- **表示項目**:
  - 📥 入力条件: `decomposed.q_in`
  - ⚡ 手順: `decomposed.q_tr`
  - 📤 出力: `decomposed.q_out`

- 構造: `decomposed.q_struct`

7. 重み調整（折りたたみパネル）

- **ラベル**: `SettingOutlined` + 「詳細な重み調整（上級者向け）」（12px）
- **内容**: WeightSlidersコンポーネント
- **デフォルト**: 折りたたみ状態

8. エラー表示

- **コンポーネント**: Alert（error、showIcon、closable）

9. 未検索時のヒント

- **アイコン**: `SearchOutlined`（48px、青色）
- **タイトル**: 「トランジションセット検索を開始」（level 5、グレー）
- **説明**: 4軸ベクトル検索の説明
- **例示**:
  - 入力条件の例
  - 手順の例
  - 出力の例
  - 構造の例

10. 検索結果

- **コンポーネント**: SearchResults
- **表示内容**: ヒット結果一覧、ローディング、エラー、検索時間

機能仕様

Props定義

```
interface TransitionSetSearchModalProps {
  open: boolean;                        // モーダル表示状態
  onClose: () => void;                  // 閉じるハンドラー
  onOpenProject?: (projectName: string) => void; // プロジェクト名クリック時のコールバック
  initialProjectName?: string;          // 初期フィルター：プロジェクト名
}
```

State管理

```
// 統合クエリ（メイン入力）
const [query, setQuery] = useState('');

// 重み
const [weights, setWeights] = useState<SearchWeights>(DEFAULT_WEIGHTS);

// AI分解結果表示用
const [decomposed, setDecomposed] = useState<DecomposedQuery | null>(null);
const [decomposing, setDecomposing] = useState(false);
```

Hooks

```
// 検索フック
const { hits, loading, error, searched, elapsedMs, search, clearResults } =
  useSemanticSearch();
```

主要機能

1. クエリをAI分解して検索リクエストを生成

```
const buildRequest = useCallback(
  async (customWeights?: SearchWeights): Promise<SemanticSearchRequest | null> => {
    if (!query.trim()) return null;
    setDecomposing(true);
    try {
      const result = await decomposeNaturalLanguageQuery(query.trim());
```

```
      setDecomposed(result);
      return {
        ...(result.q_in && { input_query: result.q_in }),
        ...(result.q_tr && { transition_query: result.q_tr }),
        ...(result.q_out && { output_query: result.q_out }),
        ...(result.q_struct && { structure_query: result.q_struct }),
        top_k: 50,
        weights: customWeights ?? weights,
      };
    } catch (err) {
      console.error('[TransitionSetSearchModal] decompose error:', err);
      // フォールバック：クエリをtransition_queryとして渡す
      return {
        transition_query: query.trim(),
        top_k: 50,
        weights: customWeights ?? weights,
      };
    } finally {
      setDecomposing(false);
    }
  },
  [query, weights]
);
```

処理フロー:

1. クエリが空でないことを確認
2. `decomposing` を `true` に設定
3. `decomposeNaturalLanguageQuery()` でAI分解を実行
4. 分解結果を `decomposed` に保存
5. 分解結果から検索リクエストを構築
6. エラー時はフォールバック（クエリ全体をtransition_queryとして使用）
7. `decomposing` を `false` に戻す

2. 通常検索

```
const handleSearch = useCallback(async () => {
  if (!query.trim()) {
    message.warning('検索クエリを入力してください');
    return;
  }
```

```
  const request = await buildRequest();
  if (!request) return;
  await search(request);
}, [query, buildRequest, search]);
```

動作:

- クエリが空の場合: 警告メッセージを表示
- `buildRequest()` で検索リクエストを構築
- `search(request)` で検索を実行

3. クイック検索

```
const handleQuickSearch = useCallback(
  async (presetKey: string) => {
    if (!query.trim()) {
      message.warning('クイック検索には検索クエリを入力してください');
      return;
    }
    const preset = WEIGHT_PRESETS[presetKey];
    if (!preset) return;

    // 重みをUIに反映
    setWeights(preset.weights);

    const request = await buildRequest(preset.weights);
    if (!request) return;
    await search(request);
  },
  [query, buildRequest, search]
);
```

動作:

1. クエリが空でないことを確認
2. プリセットキーから重みを取得
3. 重みをUIに反映（`setWeights`）
4. プリセット重みで検索リクエストを構築
5. 検索を実行

4. リセット

```
const handleClear = () => {
  setQuery('');
  setDecomposed(null);
  clearResults();
};
```

動作:

- クエリをクリア
- AI分解結果をクリア
- 検索結果をクリア

5. モーダルを閉じる

```
const handleClose = () => {
  handleClear();
  onClose();
};
```

動作:

- `handleClear()` でリセット
- `onClose()` コールバックを実行

データ型定義

SearchWeights

```
interface SearchWeights {
  w_in: number;                    // 入力条件の重み
  w_tr: number;                    // 手順の重み
```

```
  w_out: number;                    // 出力の重み
  w_struct: number;                 // 構造の重み
}
```

SemanticSearchRequest

```
interface SemanticSearchRequest {
  input_query?: string;             // 入力条件クエリ
  transition_query?: string;        // 手順クエリ
  output_query?: string;            // 出力クエリ
  structure_query?: string;         // 構造クエリ
  top_k: number;                    // 取得件数（デフォルト: 50）
  weights: SearchWeights;           // 重み
}
```

DecomposedQuery

```
interface DecomposedQuery {
  q_in?: string;                    // 入力条件
  q_tr?: string;                    // 手順
  q_out?: string;                   // 出力
  q_struct?: string;                // 構造
}
```

クイック検索プリセット

```
const QUICK_SEARCH_BUTTONS = [
  { key: 'structureFocused', icon: <ApartmentOutlined />, label: '構造重視' },
  { key: 'procedureFocused', icon: <ThunderboltOutlined />, label: '手順重視' },
  { key: 'inputFocused', icon: <InboxOutlined />, label: '入力重視' },
  { key: 'outputFocused', icon: <ExportOutlined />, label: '出力重視' },
  { key: 'balanced', icon: <StarOutlined />, label: 'バランス' },
] as const;
```

WEIGHT_PRESETS:

- 各プリセットキーに対応する重み設定
- `description`: プリセットの説明（ツールチップ表示）
- `weights`: SearchWeights

特記事項

AI分解機能

API: `decomposeNaturalLanguageQuery(query: string)`

処理:

1. 自然言語クエリを4軸に分解
2. 入力条件・手順・出力・構造を抽出
3. 分解結果を検索リクエストに変換

フォールバック:

- AI分解が失敗した場合、クエリ全体を`transition_query`として使用
- エラーはコンソールに出力

プリセット重みの適用

クイック検索時:

1. プリセット重みを取得
2. `setWeights()`でUIに反映（重み調整パネルが更新される）
3. プリセット重みで検索を実行

検索結果のプロジェクト選択

動作:

```
onOpenProject={(name) => {
  if (onOpenProject) {
```

```
    onOpenProject(name);
    handleClose();
  }
}}
```

- 検索結果からプロジェクト名をクリック
- `onOpenProject` コールバックを実行
- モーダルを閉じる
- 親コンポーネント（ProjectManagement）でプロジェクトを選択

モーダル設定

サイズ:

- 幅: `960px`
- 高さ: `maxHeight: 80vh` 、 `overflowY: auto`

破棄設定:

- `destroyOnHidden={false}` : 閉じた後もDOMを保持（検索結果を保持）

処理中の制御

```
const isProcessing = loading || decomposing;
```

非活性化対象:

- 検索実行ボタン（loading表示）
- リセットボタン
- クイック検索ボタン
- 重み調整スライダー

Enter キーの動作

TextArea:

```
onPressEnter={(e) => {
  if (!e.shiftKey) {
    e.preventDefault();
    handleSearch();
  }
}}
```

- Enter: 検索実行
- Shift+Enter: 改行

API連携

1. AI分解API

サービス: `decomposeNaturalLanguageQuery(query: string)`

レスポンス:

```
{
  q_in?: string;                    // 入力条件
  q_tr?: string;                    // 手順
  q_out?: string;                   // 出力
  q_struct?: string;                // 構造
}
```

2. セマンティック検索API

Hook: `useSemanticSearch()`

メソッド:

- `search(request: SemanticSearchRequest)`: 検索実行
- `clearResults()`: 結果クリア

レスポンス:

```
{
  hits: SemanticSearchHit[];           // ヒット結果
  loading: boolean;                    // ローディング状態
  error: string | null;                // エラーメッセージ
  searched: boolean;                   // 検索実行済みフラグ
  elapsedMs: number;                   // 検索時間（ミリ秒）
}
```

子コンポーネント

1. WeightSliders

用途: 重み調整スライダー

Props:

```
{
  weights: SearchWeights;              // 現在の重み
  onChange: (weights: SearchWeights) => void; // 重み変更ハンドラー
  disabled: boolean;                   // 非活性
}
```

2. SearchResults

用途: 検索結果一覧表示

Props:

```
{
  hits: SemanticSearchHit[];           // ヒット結果
  loading: boolean;                    // ローディング状態
  error: string | null;                // エラーメッセージ
  searched: boolean;                   // 検索実行済みフラグ
  elapsedMs: number;                   // 検索時間
  onOpenProject: (name: string) => void; // プロジェクト選択ハンドラー
}
```

3. TransitionSetBatchProcessingPanel

用途: バッチ処理パネル

Props: なし

機能: TransitionSetのバッチアップロード・処理

実装ファイル情報

パス

```
frontend/src/components/TransitionSetSearch/TransitionSetSearchModal.tsx
```

主要な依存パッケージ

- `antd` : UI コンポーネント（Modal, Tabs, Input, Button, Space, Typography, Divider, Row, Col, Badge, Collapse, Tooltip, Alert, message）
- `@ant-design/icons` : アイコン（SearchOutlined, SettingOutlined, CloudUploadOutlined, ThunderboltOutlined, AccountBookOutlined, InboxOutlined, ExportOutlined, ApartmentOutlined, StarOutlined）
- `react` : React本体（useState, useCallback）

Hooks

- `useSemanticSearch` : セマンティック検索フック

サービス

- `semanticSearchService` : AI分解API

子コンポーネント

- `WeightSliders`：重み調整スライダー
- `SearchResults`：検索結果一覧
- `TransitionSetBatchProcessingPanel`：バッチ処理パネル

変更履歴

バージョン	日付	変更内容	担当者
1.0	2026-03-11	実装ベースで仕様書を新規作成（実装ファイル: TransitionSetSearchModal.tsx）	Cline

---

関連ドキュメント

- プロジェクト管理画面
- 検索結果一覧
- 重み調整スライダー
- バッチ処理パネル